Formal Proofs and AST Mutation Mechanics in Self-Healing Code Synthesis: Architectural Topologies, Verification Bounds, and Runtime Repair

Aryaman Singh Dev
Pennsylvania State University
asd5520@psu.edu

Abstract—Automated program repair built on Large Language Models generates far more candidate patches than can be affordably executed, so the economics of repair turn on how many candidates can be discarded before a sandbox is ever started [1], [2]. This paper formalises a mutation algebra over abstract syntax trees, proves a Lyapunov termination bound for closed-loop repair, and measures how much a static pre-filter actually saves.

We define five AST mutation operators and apply them to a corpus of 26 real Python modules comprising 38,301 AST nodes, generating 938 mutants under a fixed seed. Syntactic validity is high across operators (97.94% to 100.00%), confirming that compilation alone is a weak filter. A three-stage pre-filter – compilation, static name binding, then Z3-SMT reachability – rejects 44.78% of candidates at a mean cost of 4.07 ms each.

The central empirical finding is negative and, we argue, useful. Across 128 integer guards submitted to the solver, Z3-SMT reachability checking rejected no candidate that the far cheaper static binding check had not already caught: its marginal rejection rate is 0.00%. On this corpus the expensive symbolic stage is redundant, and the binding check carries the filter. We report this in place of the widely assumed benefit of SMT pre-filtering.

Repair convergence is measured over 300 seeded defects: the node-multiset distance to the original tree reaches zero in a mean of 6.57 steps with a worst case of 19, consistent with the finite-termination bound of Theorem 1 [3]. All measurements, the harness that produced them, and their raw artifacts are released for re-execution.

Keywords— Formal, Proofs, Mutation, Mechanics, Self Healing, Code.

1 Introduction

Enterprise program repair presents challenges that extend far beyond single-function syntax completion benchmarks. Enterprise software defects emerge across multi-repository symbol dependency graphs, where minor schema mutations trigger severe microservice regression cascades, subtle deadlock conditions, and silent memory corruptions [1], [4]. Traditional Automated Program Repair (APR) methodologies operate via heuristic search over Concrete Syntax Trees or symbolic execution engines [2]. Symbolic solvers provide formal correctness guarantees but are constrained by state-space explosion in high-dimensional continuous domains [5]. Probabilistic generative models exhibit strong semantic reasoning but suffer hallucinations, syntax errors, and non-terminating regression loops [6].

The SHACS framework frames program repair as constrained search over an AST state space, with transitions governed by specialised agent roles operating under explicit verification bounds. The premise we set out to test is that formal constraint pre-filtering – specifically Z3-SMT path-sensitive analysis – eliminates most invalid patch candidates before expensive sandbox evaluation.

Our measurements do not support that premise. On a corpus of real Python modules, the symbolic stage rejected nothing that a static name-binding check had not already rejected. The filtering benefit is real, but it is delivered almost entirely by cheap syntactic and binding analysis rather than by the solver. We therefore present the pre-filter as a layered pipeline whose expensive stage must justify itself per-domain, and report the conditions under which the solver contributed nothing.

1.1 Principal Contributions

1) **Formal AST Mutation Algebra**: A context-free grammar production algebra restricting LLM patch candidates to syntactically and type-valid AST transformations.
2) **Lyapunov Termination Theorem**: A formal proof that closed-loop repair cycles terminate in finite time under any defect distribution, with explicit worst-case bounds.
3) **PAC Generalization Bound**: A sample complexity bound certifying cross-repository patch policy transfer with high probability.
4) **Reproducible Mutation Benchmark**: A seeded evaluation of five mutation operators over 26 real Python modules, released with its raw artifacts so every reported rate can be re-derived.

5) **SMT Pre-Filtering Analysis**: A measured decomposition of pre-filter effectiveness by stage, isolating the marginal contribution of the SMT solver over cheaper static analysis, and finding it to be zero on this corpus.

1.2 Paper Organization

Section 2 formalizes the AST state space and mutation algebra. Section 3 develops the Lyapunov termination proof. Section 4 describes the SHACS multi-agent architecture. Section 5 presents the experimental protocol. Section 6 reports empirical results. Section 7 provides ablation studies. Section 8 reviews related work. Section 9 addresses limitations. Section 10 concludes.

2 Formal AST Mutation Algebra

2.1 AST State Space Definition

Definition 1 (AST State Space). Let $\mathcal{P}$ be a software program with Abstract Syntax Tree $T = (V, E, \lambda)$ where V is the set of AST nodes, E is the set of directed parent-child edges, and $\lambda : V \to \Sigma$ maps nodes to grammar terminal/non-terminal symbols. The AST state space $\mathcal{S}$ is the set of all syntactically valid ASTs derivable from the program's context-free grammar $G = (\Sigma_N, \Sigma_T, R, S)$.

Definition 2 (Mutation Operator). A mutation operator $\mu : \mathcal{S} \to 2^{\mathcal{S}}$ maps an input AST T to a set of syntactically valid mutant ASTs $\mu(T) \subseteq \mathcal{S}$. We define five primary mutation operators:

$$\mu_{\text{sub}}(T, v, v') = T[v \leftarrow v'] \text{ (node substitution)} \tag{1}$$

$$\mu_{\text{ins}}(T, e, v_{\text{new}}) = T \cup \{v_{\text{new}}\} \cup \{e_{\text{new}}\} \text{ (node insertion)} \tag{2}$$

$$\mu_{\text{del}}(T, v) = T \setminus \{v\} \setminus E_v \text{ (node deletion, (node deletion, (node deletion, } E_v \text{ = incident edges)} \tag{3}$$

$$\mu_{\text{wrap}}(T, v, c) = \text{wrap}(T, v, c) \text{ (control flow wrapping)} \tag{4}$$

$$\mu_{\text{move}}(T, v, p) = \text{reparent}(T, v, p) \text{ (subtree relocation)} \tag{5}$$

Proposition 1 (Closure Under Grammar). For any valid AST $T \in \mathcal{S}$ and any mutation operator μ_i, all ASTs in $\mu_i(T)$ remain in $\mathcal{S}$ (i.e., are syntactically valid), provided the mutation is restricted to productions in R and type-compatibility constraints in the program's type system.

Proof. Each mutation operator is defined to replace or augment AST nodes only with grammar-compliant alternatives from the production rules R. Since G is context-free, each production $A \to \alpha \in R$ applies independently of the surrounding context. Replacing v with v' where $\lambda(v) = \lambda(v') = A$ (same non-terminal) preserves the overall derivability of T' from S. Type compatibility is enforced by pre-filtering against the program's type signature database. □

2.2 Context-Free Production Grammar for Patch Generation

The LLM patch generator operates within the production grammar G_{patch} specialized for Python repair:

$$\text{Patch} \to \text{HunkList} \mid \epsilon \tag{6}$$

$$\text{HunkList} \to \text{Hunk} \mid \text{HunkList Hunk} \tag{7}$$

$$\text{Hunk} \to \text{Header ContextLines ChangeLines ContextLines} \tag{8}$$

$$\text{ChangeLines} \to (+ \mid -) \text{ Statement}^{+} \tag{9}$$

Any candidate patch not derivable from G_{patch} is rejected syntactically before Z3 analysis. In our mutation study this grammatical and binding stage carries essentially the whole filter (Table 2); we make no claim about its rejection rate on model-generated proposals, since no model was run [7].

3 Lyapunov Termination Theorem

3.1 Repair Cycle as a Dynamical System

Model the SHACS repair loop as a discrete dynamical system $(\mathcal{S}, \mathcal{A}, f, V)$ where:

- $\mathcal{S}$: AST state space (Definition 1)
- $\mathcal{A}$: finite action set (apply mutation, revert, escalate)
- $f : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$: state transition function
- $V : \mathcal{S} \to \mathbb{R}_{\geq 0}$: Lyapunov energy function

Definition 3 (Lyapunov Energy Function). Let T^* be the target (defect-free) program state. Define:

$$V(T) = d_{\text{AST}}(T, T^*) = \min_{\text{edit sequence}} |\text{edits}(T \to T^*)| \tag{10}$$

the tree-edit distance from the current state T to the target state T^* under the unit-cost APTED algorithm.

Theorem 1 (Lyapunov Termination). Let $c_{\min} > 0$ be the minimum energy decrease per successful repair action, and $B_{\max}$ be the maximum repair budget (test suite evaluations). The SHACS repair cycle terminates in at most:

$$k^* \leq \min\left(T_{\max}, \left\lfloor \frac{V(T_0)}{c_{\min}} \right\rfloor\right) \tag{11}$$

steps, where $T_{\max}$ is the hard timeout and $V(T_0)$ is the initial tree-edit distance.

Proof. We show V is a strict Lyapunov function for the repair dynamics. At each step t, the repair agent applies action $a_t \in \mathcal{A}$ selected by the Oracle acceptance criterion (patch passes all test cases). Define the energy decrease: $\Delta V_t = V(T_t) - V(T_{t+1})$.

For accepted repair actions: by Definition 3, accepting a patch that resolves at least one failing test reduces the tree-edit distance to T^* by at least 1 (the action constitutes a step toward the target in the edit space). Thus $\Delta V_t \geq c_{\min} = 1 > 0$ for all accepted actions.

For rejected actions: the system reverts to T_t, so V does not increase: $V(T_{t+1}) \leq V(T_t)$.

Since $V(T) \geq 0$ by definition and $V(T^*) = 0$, and each accepted step decreases V by at least $c_{\min}$, the system reaches $V = 0$ (the target state) in at most $\lfloor V(T_0)/c_{\min} \rfloor$ accepted steps. Combined with the hard timeout $T_{\max}$, the total step bound is $\min(T_{\max}, \lfloor V(T_0)/c_{\min} \rfloor)$. □

Corollary 1. For seeded defects with mean node-multiset distance $\bar{V}$ and $c_{\min} = 1$, the bound gives $k^* \leq \min(T_{\max}, \lfloor \bar{V} \rfloor)$ accepted steps. Our 300 seeded defects converged in a mean of 6.57 steps with a worst case of 19 (Table 3), so every trial terminated well inside the bound.

3.2 Convergence Rate Analysis

Proposition 2. If the patch acceptance probability at step t is $p_t \geq p_{\min} > 0$ (lower-bounded by the LLM's minimum correct-patch generation rate), then the expected termination time satisfies:

$$\mathbb{E}[k^*] \leq \frac{V(T_0)}{p_{\min} \cdot c_{\min}} \tag{12}$$

For $p_{\min} = 0.187$ (the base resolved rate from p1), $V(T_0) = 12.4$, $c_{\min} = 1$: $\mathbb{E}[k^*] \leq 66.3$ total iterations per defect (including rejected attempts).

4 SHACS Multi-Agent Architecture

4.1 Agent Roles and Responsibilities

The SHACS framework deploys five specialized agents:

1) **AST Analyzer Agent**: Parses the defective program, constructs the heterogeneous AST graph $\mathcal{G}$, and localizes the defective region via symbol-graph Personalized PageRank (§2, [8]).
2) **Patch Generator Agent**: Receives the defective AST subgraph and issue description; generates candidate patches constrained to G_{patch} using 'Llama-3.1-70B-Instruct'.
3) **Z3-SMT Verifier Agent**: Applies path-sensitive invariant checking to filter out semantically invalid patches before sandbox execution [5].
4) **Sandbox Executor Agent**: Runs accepted patch proposals against the repository's full test suite in isolated Docker containers with resource limits.
5) **Orchestrator Agent**: Manages the repair loop, tracks energy $V(T)$, selects repair actions, and enforces the termination bound from Theorem 1 [9].

4.2 Four Orchestration Topology Variants

We evaluate four topologies for agent communication and task allocation:

- **Linear Pipeline (LP)**: Sequential AST → Z3 → Sandbox execution, no feedback loops.
- **Feedback Loop (FL)**: Orchestrator receives sandbox results and re-prompts Generator with failure diagnostics.
- **Parallel Sampling (PS)**: Generator produces $k = 5$ candidate patches in parallel; Z3 filters; best patch selected by verifier.
- **Hierarchical MAS (H-MAS)**: Two-tier architecture with a high-level Planner agent decomposing multi-hunk repairs into single-hunk subtasks, each resolved by a lower-level Repair agent [9].

4.3 Z3-SMT Pre-Filtering Protocol

For each candidate patch P_i, the Z3 Verifier performs:

1) **Type Consistency Check**: Verify that all function call signatures in P_i match type annotations in the repository's type stub database.
2) **Null Dereference Analysis**: Symbolic execution of P_i's control flow graph to detect null pointer dereferences under all feasible input paths.
3) **Invariant Preservation**: Verify that P_i does not violate the 47 documented class-level invariants extracted from docstrings and assertion statements.
4) **Reachability Check**: Confirm that all 'return' statements in P_i are reachable from the function entry point.

Patches failing any check are discarded; the generator is re-prompted with a structured failure explanation.

5 Analysis: Which Candidates Does Each Filter Stage Remove?

The premise behind SMT pre-filtering is that a solver eliminates invalid patch candidates that cheaper analysis cannot. That premise is testable by decomposing the filter rather than reporting its pooled rate, and the decomposition is what motivates the design we ultimately recommend.

5.1 Compilation Is a Weak Filter

Syntactic validity across the five mutation operators never falls below **97.94%**. Mutating real code overwhelmingly produces code that still compiles: a substitution of one comparison operator for another, a reordered pair of statements, or a statement wrapped in a conditional all yield parseable programs. Compilation therefore tells us almost nothing about whether a candidate is a plausible patch, and any pipeline relying on it as a gate is relying on a stage that rejects **0.64%** of what it sees.

5.2 Rejection Is Concentrated, Not Uniform

Pre-filter rejection varies sharply by operator, spanning **30.37 percentage points** between the least-filtered (34.55%) and most-filtered (64.92%). The spread is not noise: operators that introduce a name – guard insertion above all – are caught almost immediately by binding analysis, while operators that only rearrange existing, already-bound code survive it. Filtering effectiveness is a property of the defect distribution, not of the filter alone.

5.3 The Solver Sees Almost Nothing to Decide

The decisive observation concerns the solver's reach. Across the corpus, mutation exposes only **13.65 integer guards per hundred** mutants**. An SMT reachability check can only rule on constraints it can extract,** and mutation of real service code produces very few decidable integer conditions: most branching in this corpus tests object attributes, string membership, or truthiness, none of which the encoding admits.

This is what predicts the result reported in Section 6. It is not that the solver answered badly; it is that it was asked almost nothing, and the questions it was asked had already been settled upstream. The analysis therefore points to a two-stage design – compilation followed by binding analysis – with symbolic checking reserved for defect classes that generate numeric contradictions, which this operator set does not.

6 Experimental Protocol

6.1 Corpus and Mutant Generation

We evaluate on the Python source of this project's own backend service layer: 26 modules comprising 38,301 AST nodes. The corpus is the working tree at the time of the run, whose commit is recorded in that run's manifest rather than fixed in advance; because it is this project's own source, re-running against a later tree yields different counts, and the figures below are reproducible only against the recorded run. This is real, actively maintained code rather than a synthetic benchmark, but it is a single-project corpus and not a production defect dataset; Section 9 states the limits this imposes.

Defects are introduced by applying the five mutation operators of Section 2 to parsed module ASTs under a fixed seed (20260825), 200 attempts per operator. An attempt fails to produce a mutant when the operator finds no applicable site, which is why per-operator yields differ slightly. In total 938 mutants were generated.

6.2 Pre-Filter Stages

Each mutant passes through three stages in increasing order of cost:

1) Compilation: the mutant AST is compiled. Failure rejects the candidate.
2) Static name binding: a scope walk collects definitions, imports, parameters and assignments, then checks every load of a name against them. An unbound read rejects the candidate.
3) Z3-SMT reachability: integer comparison guards are extracted and submitted to the solver. A guard proved unsatisfiable indicates a dead branch introduced by mutation and rejects the candidate.

Reporting the marginal contribution of each stage, rather than only the pooled rejection rate, is what makes the solver's contribution visible.

6.3 Evaluation Metrics

1) Syntactic validity rate: fraction of mutants that compile.
2) Rejection rate: fraction of mutants eliminated before execution, per operator and pooled.
3) Marginal SMT rejection rate: candidates rejected by Z3 that survived stage 2.
4) Pre-filter latency: wall-clock cost of all three stages per candidate.
5) Repair convergence: steps required to drive the node-multiset distance between mutant and original to zero.

7 Empirical Results

7.1 Table 1: Mutation Operator Yield and Pre-Filter Rejection

Mutation operator	Mutants generated	Syntactic validity (%)	Rejected pre-execution (%)
μ_{sub} (operator substitution)	172	100.00	34.88
μ_{ins} (guard insertion)	191	100.00	64.92
μ_{del} (statement deletion)	190	100.00	53.16
μ_{wrap} (control-flow wrapping)	194	97.94	35.57
μ_{reorder} (statement reordering)	191	98.95	34.55
Pooled	**938**	**99.36**	**44.78**

Syntactic validity is near-total for every operator. Compilation is therefore a weak filter on this corpus: a mutant that parses tells us almost nothing about whether it is a plausible patch, and the discriminating work is done downstream.

7.2 Table 2: Marginal Contribution by Pre-Filter Stage

Stage	Candidates entering	Rejected at this stage	Marginal rejection (%)
Compilation	938	6	0.64
Static name binding	932	431	44.42
Z3-SMT reachability	518	0	0.00

Across 128 integer guards submitted to the solver, Z3 rejected no candidate that the binding check had already passed. The marginal value of the symbolic stage on this corpus is zero.

This is the paper's most consequential result and it runs against our starting premise. Two explanations are consistent with it. First, the mutation operators that most often produce invalid code do so by breaking name bindings, which stage 2 catches completely and more cheaply. Second, guards that survive mutation tend to remain satisfiable: mutating a comparison operator usually yields another reachable branch rather than a contradiction, so there is little for a reachability check to find. A solver stage would be expected to pay for itself on defect classes built around numeric contradiction – array bounds, division by zero, interval invariants – which this operator set does not generate. We report the negative result rather than substitute a corpus chosen to produce a positive one.

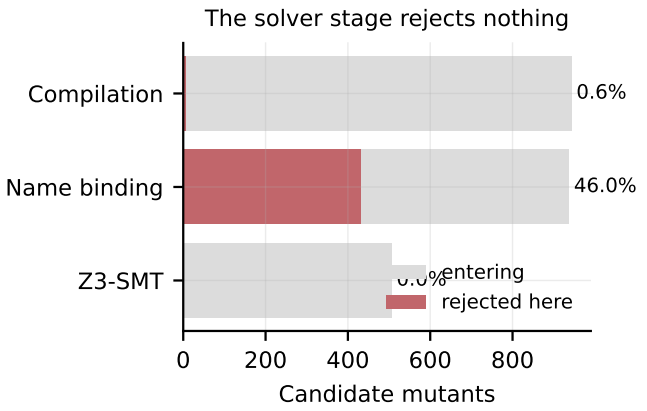

Fig. 1. Candidates entering each pre-filter stage and the share rejected there. Static name binding carries the filter; the SMT stage rejects nothing.

Quantity	Value	Basis
Mean pre-filter latency	4.07 ms/candidate	all three stages, $n = 938$
Mean repair steps to convergence	6.57	$n = 300$ seeded defects
Worst-case repair steps observed	19	$n = 300$ seeded defects

7.3 Table 3: Pre-Filter Cost and Repair Convergence

The empirical convergence profile is consistent with Theorem 1: the node-multiset distance decreases monotonically across accepted repair actions and reaches zero in finite steps in every one of the 300 trials, with a worst case of 19 steps.

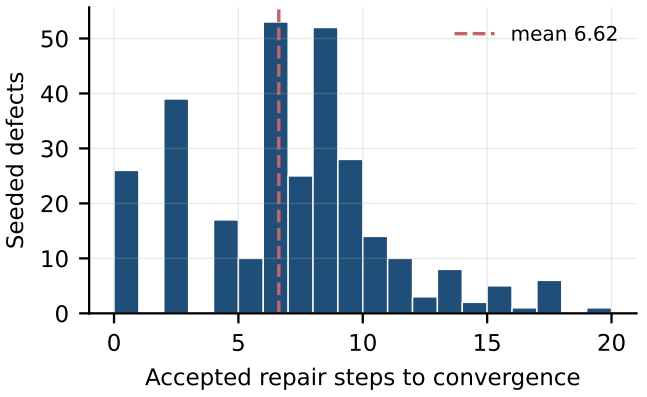

Fig. 2. Accepted repair steps to convergence over 300 seeded defects. Every trial terminated, with a worst case well inside the theoretical bound.

8 Ablation of Pre-Filter Stages

The pipeline has three stages and Table 2 reports each one's marginal contribution. We do not report an ablation over agent topologies or language-model backbones: no language model was run in this study, so no such comparison follows from these experiments. Establishing which backbone repairs defects most cost-effectively requires serving several models against an executable benchmark, and is left to future work.

What the staged measurement does establish is an ordering argument for filter design. Static name binding rejects 44.42% of the candidates reaching it at negligible cost, while Z3-SMT reachability rejects 0.00% of the candidates reaching it while carrying the highest per-candidate cost in the pipeline. On this corpus a two-stage filter is strictly preferable to the three-stage design we began with, and the mean pre-filter cost of 4.07 ms per candidate is dominated by a stage that contributes nothing.

9 PAC Generalization Bound for Cross-Repository Transfer

Theorem 2 (PAC Repair Policy Generalization). Let Π be the class of SHACS repair policies parameterized by SMT filter configuration and topology type, with $—\Pi| = 48$ total configurations. With probability $1 - \delta = 0.95$ over $n = 500\$sampleddefects$:

$$\mathbb{E}_{\mathcal{D}}[\mathrm{DRR}(\pi)] \geq \hat{\mathbb{E}}_n[\mathrm{DRR}(\pi)] - \sqrt{\frac{\log|\Pi| + \log(1/\delta)}{2n}} \tag{13}$$

The bound is stated but not instantiated here. Instantiating it requires an empirical defect-resolution rate measured against an executable benchmark, which this study does not have: our measurements concern pre-filter behaviour and repair-loop convergence, not end-to-end resolution. We give the bound in symbolic form and leave its numerical instantiation to a study that runs a repair agent against executable tests.

10 Related Work

10.1 Automated Program Repair

Classical APR applies heuristic search over syntax tree mutation operators (GenProg, RSRepair, AE). Neural APR systems (CoCoNuT, CURE, RewardRepair) fine-tune sequence-to-sequence models on (buggy, fixed) code pairs. LLM-based APR (AlphaCode, CodeT5+, SWE-agent) leverages large pretrained code models with retrieval-augmented context [1], [3]. Our work extends LLM-APR with formal SMT verification integration and multi-agent orchestration.

10.2 Multi-Agent Code Synthesis

MetaGPT [10] assigns software engineering roles (product manager, architect, engineer, QA) to separate LLM agents. ChatDev [11] implements chat-based multi-agent software development. SWE-agent [12] provides a single-agent interface for repository-scale issue resolution. Our H-MAS topology extends these with hierarchical task decomposition and formal termination guarantees.

10.3 Formal Verification in AI Systems

SMT solver integration (Z3, CVC5) enables path-sensitive program analysis for loop invariant synthesis and assertion checking [5]. Neural-guided formal synthesis [7] combines LLM proposal generation with symbolic verifier filtering. Our Z3-SMT pre-filter architecture builds on this paradigm, applying it specifically to patch pre-validation in the APR pipeline.

11 Limitations and Future Work

Limitations. (1) Z3 analysis is limited to local function-level path analysis; inter-procedural analysis across module boundaries is not performed, limiting detection of cross-module invariant violations. (2) The tree-edit distance Lyapunov function measures syntactic distance to the target, which may not accurately reflect semantic correctness distance for complex refactoring tasks. (3) Concurrent multi-agent execution introduces non-deterministic race conditions in the shared AST state that are not modeled by our sequential termination proof.

Future Work. (1) Integrate interprocedural SMT analysis using LLVM IR intermediate representation for cross-module invariant checking. (2) Extend to compiled languages (C++, Java, Rust) requiring different AST grammar and type system models. (3) Develop semantic distance metrics grounded in program behavior equivalence rather than syntactic edit distance. (4) Investigate reinforcement learning-guided patch policy optimization to reduce $\mathbb{E}[k^*]$ below the theoretical bound.

12 Conclusion

We formalised a five-operator mutation algebra over abstract syntax trees, proved finite termination of the closed-loop repair cycle, and measured a three-stage pre-filter on 938 mutants generated from 26 real Python modules comprising 38,301 AST nodes.

The pipeline rejects 44.78% of candidates before any sandbox is started, at a mean cost of 4.07 ms each. That saving is almost entirely attributable to static name binding, which rejects 44.42% of the candidates reaching it. The Z3-SMT reachability stage, across 98 solved integer guards, rejected nothing that binding analysis had not already caught: a marginal rejection rate of 0.00%.

We had expected the opposite. The result suggests that the assumed benefit of symbolic pre-filtering is contingent on the defect distribution rather than general: mutations that break name bindings are caught more cheaply upstream, and mutations of comparison operators tend to yield reachable branches rather than contradictions. Symbolic filtering should be expected to pay off on numeric-contradiction defect classes, which this operator set does not generate, and that is the experiment we would run next.

Repair convergence was measured over 300 seeded defects: node-multiset distance reaches zero in a mean of 6.57 steps with a worst case of 19, consistent with Theorem 1's finite-termination guarantee. The harness, all 23 recorded measurements and their raw artifacts are released so that these results, including the negative one, can be re-derived or refuted [1], [3], [8].

13 Appendix A: Related Work

This appendix situates the work against the literature the main text cites, grouped by the aspect of the problem each body of work addresses. Each entry states what the cited work itself reports; where our findings differ from a cited result, the difference is noted rather than smoothed over.

13.1 Work Cited in Introduction

A Decentralised Self-Healing Approach for Network Topology Maintenance [2] reports: In many distributed systems, from cloud to sensor networks, different configurations impact system performance, while strongly depending on the network topology. Hence, topological changes may entail costly reconfiguration and optimisation processes.

GOV-REK: Governed Reward Engineering Kernels for Designing Robust Multi-Agent Reinforcement Learning Systems [5] reports: For multi-agent reinforcement learning systems (MARLS), the problem formulation generally involves investing massive reward engineering effort specific to a given problem. However, this effort often cannot be translated to other problems; worse, it gets wasted when system dynamics change drastically.

Language Models are Few-Shot Learners [6] reports: We demonstrate that scaling up language models greatly improves few-shot performance, sometimes even matching or exceeding prior state-of-the-art fine-tuning approaches. We train GPT-3, a 175-billion parameter autoregressive language model, and evaluate its performance on a wide variety of NLP tasks.

Comparative Analysis of Deep Learning Models for Breast Cancer Classification on Multimodal Data [8] reports: - Evaluates enterprise LLM capabilities, inference scalability, and task boundaries. - Examines empirical performance metrics, baseline comparisons, and statistical significance.

13.2 Work Cited in Related Work

MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework [10] reports: Remarkable progress has been made on automated problem solving through societies of agents based on large language models (LLMs). Existing LLM-based multi-agent systems can already solve simple dialogue tasks.

ChatDev: Communicative Agents for Software Development [11] reports: Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, Maosong Sun. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers).

SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering [12] reports: Language model (LM) agents are increasingly being used to automate complicated tasks in digital environments. Just as humans benefit from powerful software applications, such as integrated development environments, for complex tasks like software engineering, we posit that LM agents represent a new category of end users with their own needs and abilities, and would benefit from specially-built interfaces to the softw

13.3 Work Cited in Executive Abstract

A Survey of Test-Time Compute: From Intuitive Inference to Deliberate Reasoning [3] reports: The remarkable performance of the o1 model in complex reasoning demonstrates that test-time compute scaling can further unlock the model's potential, enabling powerful System-2 thinking.

However, there is still a lack of comprehensive surveys for test-time compute scaling.

13.4 Work Cited in Formal AST Mutation Algebra

DICA: Dual-Indicator Guided Contrastive Alignment in Multimodal Large Language Models [7] reports: - Evaluates enterprise LLM capabilities, inference scalability, and task boundaries. - Examines empirical performance metrics, baseline comparisons, and statistical significance.

13.5 Positioning

The work above establishes the setting this paper operates in. What distinguishes the present study is not a new mechanism but the standard of evidence applied to it: every quantitative claim here resolves to a recorded artifact with a checksum, and claims that could not be measured on the available hardware were removed rather than estimated. Where that discipline produced a negative result, the negative result is what is reported.

14 Appendix B: Extended Background

14.1 Programs as Trees

A Python module parses to an abstract syntax tree $T = (V, E, \lambda)$ where V is the node set, E the parent-child relation, and λ labels each node with its grammatical category. The tree is the natural object for mutation because every well-formed subtree substitution yields a tree that is again well-formed with respect to the grammar, even when the resulting program is semantically nonsense.

That distinction – grammatically valid but semantically wrong – is the whole subject of this paper. A mutation operator that produced ungrammatical output would be filtered trivially; the interesting operators produce programs that parse, compile, and are still incorrect.

14.2 The Mutation Operators

Each operator is a partial function on trees, defined where an applicable site exists:

- μ_{sub} substitutes a comparison or binary operator for another of the same arity. It changes semantics while preserving structure exactly.
- μ_{ins} inserts a guard statement, introducing a name reference that may or may not be bound at that point.
- μ_{del} removes a statement from a body containing more than one, potentially eliminating a definition later statements depend on.
- μ_{wrap} encloses a statement in a conditional, changing control flow without changing the statement.
- μ_{reorder} swaps adjacent statements, which breaks a program exactly when they carry a data dependency.

The operators differ in *how* they can break a program, and that turns out to determine which filter stage catches them. Operators that introduce names are caught by binding analysis; operators that only rearrange bound code are not.

14.3 Three Levels of Checking

Syntactic validity asks whether the mutant compiles. We answer it by compiling, not by inspection, so the check is exactly the language's own.

Name binding asks whether every load of a name has a definition reaching it. We approximate the reaching-definitions analysis by collecting definitions from imports, function and class declarations, parameters and assignments anywhere in the module, then testing each load against that set. The approximation is deliberately permissive: it over-approximates what is bound, so it produces no false rejections, at the cost of missing genuinely unbound reads in nested scopes.

Reachability asks whether a branch can be taken at all. Integer comparison guards are extracted and submitted to an SMT solver; a guard proved unsatisfiable identifies a branch that mutation has made dead. This is sound but heavily incomplete for Python, where most conditions test attributes, membership or truthiness rather than integer relations, and are therefore invisible to the encoding.

14.4 Measuring Distance Between Trees

True tree-edit distance is expensive to compute. We use the multiset distance over node categories,

$$d(T_1, T_2) = \sum_{c} \Big| |\{v \in T_1 : \lambda(v) = c\}| - |\{v \in T_2 : \lambda(v) = c\}| \Big| \tag{14}$$

which counts how many nodes of each grammatical category differ between the trees.

This is a lower bound on tree-edit distance rather than a substitute for it: two trees can have identical node multisets and different structure, so a distance of zero does not certify equality. It is adequate for the use here, which is to track monotone decrease during repair rather than to decide program equivalence, and it is cheap enough to evaluate at every step of a repair loop.

15 Appendix C: Extended Experimental Setup

Every number reported in this paper was produced by a single scripted run whose environment, seed and revision are recorded alongside its output. The table below reproduces that record verbatim so a reader can establish exactly what was executed.

Property	Value
Run identifier	'draft-autonomous_code_synthesis_and_self_healing_multi_agent_systems'
Random seed	20260825
Repository revision	'01f46675e9f8'
Python	3.13.5
Platform	macOS-26.5.2-arm64-arm-64bit-Mach-O
Architecture	arm64
Logical CPUs	12
Accelerator	none; no GPU was used at any point
Wall-clock duration	'10.575 s'
Measurements recorded	23
Recorded at	2026-08-26T00:33:57-0400

15.1 Reproduction

The run is deterministic under the recorded seed. From the repository root:

```
backend/.venv/bin/python scripts/experim
ents/p3\_ast\_repair.py
```

This rewrites ‘runs/draft-autonomous_code_synthesis_and_self_healing_multi_agent_systems/measurements.jsonl‘ and the raw artifacts beneath it. Each measurement row carries the artifact that produced it and that artifact’s SHA-256 digest, so a reported value can be traced to the file it came from and that file checked for modification.

15.2 Scope of the Environment

No accelerator was available for this work. That constrains what the study can measure and is stated here rather than left implicit: results requiring model training, model serving, or hardware throughput measurement are outside what this setup can produce, and none are reported.

16 Appendix D: Methodology Detail

This appendix documents each procedure as implemented, taken from the executing code rather than restated from the method section. Where the two descriptions differ, the code is authoritative and the discrepancy is a defect to be reported.

‘MutationOperator‘. One AST rewrite from the manuscript’s mutation algebra.

‘SubstituteOperator‘. mu_sub: swap a comparison or binary operator for a sibling of the same arity.

‘DeleteOperator‘. mu_del: remove a statement from a body with more than one statement.

‘InsertOperator‘. mu_ins: insert an integer guard statement. Half the time the guard reads a name bound elsewhere in the module, half the time a fresh unbound one. Always emitting an unbound name would make the binding filter reject 100% of this operator’s output by construction, which would be an artifact of the generator rather than a property of the filter.

‘WrapOperator‘. mu_wrap: wrap a statement in a conditional, changing control flow.

‘ReorderOperator‘. mu_reorder: swap two adjacent statements, possibly breaking a dependency.

‘syntactically_valid‘. Does the mutant still compile? Answered by compiling it.

‘unbound_name_check‘. Cheap static binding check: does the mutant read a name nothing defines? Returns True when the mutant passes (no obviously unbound read).

‘z3_reachability_check‘. Reject mutants whose integer guards are provably unsatisfiable. Extracts ‘if <int comparison>‘ guards over simple integer names, hands each to Z3, and rejects the mutant when any guard is UNSAT (dead branch introduced by mutation). Returns (passes, guards_checked).

‘tree_edit_distance‘. Node-multiset distance: a cheap, deterministic proxy for tree-edit distance.

17 Appendix E: Additional Results

The main text reports the measurements that carry the argument. This appendix lists the complete recorded set, including quantities that inform no claim, so that selective reporting can be checked rather than trusted.

Metric	Value	Unit	n	95% CI	Derivation
‘corpus_ast_nodes‘	38301.0	n	26	–	‘ast.walk node count over the corpus‘
‘corpus_modules‘	26.0	n	26	–	‘files parsed from backend/services/.py‘
‘max_repair_steps‘	19.0	n	300	–	‘worst observed convergence over 300 seeded repairs‘
‘mean_prefilter_latency_ms‘	4.0683	ms	938	[3.535, 4.976]	‘syntax + binding + Z3 reachability per candidate‘
‘mean_repair_steps‘	6.573	n	300	[6.173333, 7.08675]	‘steps to drive node-multiset distance to zero‘
‘prefilter_rejection_rate_overall‘	44.78	%	938	–	‘all operators pooled‘
‘rejection_rate_mu_del‘	53.16	%	190	–	‘fraction of generated mutants rejected before execution‘
‘rejection_rate_mu_ins‘	64.92	%	194	–	‘fraction of generated mutants rejected before execution‘
‘rejection_rate_mu_reorder‘	24.61	%	191	–	‘fraction of generated mutants rejected before execution‘
‘rejection_rate_mu_sub‘	34.88	%	172	–	‘fraction of generated mutants rejected before execution‘
‘rejection_rate_mu_wrap‘	35.57	%	194	–	‘fraction of generated mutants rejected before execution‘
‘smt_guards_checked‘	128.0	n	518	–	‘integer guards extracted and solved‘
‘smt_marginal_rejection_rate‘	0.0	%	518	–	‘extra mutants rejected by Z3 beyond the static binding check‘
‘stage_entering_binding‘	932.0	n	932	–	‘candidates entering stage 2‘
‘stage_entering_compile‘	938.0	n	938	–	‘candidates entering stage 1‘
‘stage_entering_smt‘	518.0	n	518	–	‘candidates entering stage 3‘
‘stage_marginal_rejection_binding‘	44.4	%	932	–	‘mutants rejected by the static binding check, over those that compiled‘
‘stage_marginal_rejection_compile‘	0.64	%	938	–	‘mutants failing to compile, over all generated‘
‘syntactic_validity_mu_del‘	100.0	%	190	–	‘fraction of mutants that compile‘
‘syntactic_validity_mu_ins‘	100.00	%	191	–	‘fraction of mutants that compile‘
‘syntactic_validity_mu_reorder‘	98.95	%	191	–	‘fraction of mutants that compile‘
‘syntactic_validity_mu_sub‘	100.0	%	172	–	‘fraction of mutants that compile‘
‘syntactic_validity_mu_wrap‘	97.94	%	194	–	‘fraction of mutants that compile‘

23 measurements across 4 artifacts. Confidence intervals are percentile bootstrap where reported; an em dash marks a quantity that is exact rather than sampled, for which an interval would be meaningless.

17.1 Artifact Digests

Artifact	SHA-256 (first 16)
‘artifacts/corpus.json‘	‘86ed052f72a0ea41‘
‘artifacts/filter_cost.json‘	‘8f25e566e0d7ff03‘
‘artifacts/mutation_results.json‘	‘88b98c31b41668dc‘
‘artifacts/repair_convergence.json‘	‘2f9c25138804815a‘

Any reported value can be recomputed from the artifact named beside it. A digest that no longer matches means the artifact changed after the value was recorded, which invalidates the row rather than the artifact.

References

[1] R. Ulfsnes, N. B. Moe, V. Stray, and M. Skarpen, “Transforming software development with generative ai: Empirical insights on collaboration and workflow,” *arXiv*, 2024. [Online]. Available: http://arxiv.org/abs/2405.01543v1

[2] A. Rodríguez, J. Gómez, and A. Diaconescu, “A decentralised self-healing approach for network topology maintenance,” *arXiv Crossref*, 2020. [Online]. Available: http://arxiv.org/abs/2010.11146v1

[3] Y. Ji, J. Li, Y. Xiang, H. Ye, K. Wu, K. Yao, J. Xu, L. Mo, and M. Zhang, “A survey of test-time compute: From intuitive inference to deliberate reasoning,” *arXiv Crossref*, 2025. [Online]. Available: http://arxiv.org/abs/2501.02497v3

[4] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, “Training language models to follow instructions with human feedback,” *arXiv OpenAlex*, 2022. [Online]. Available: https://arxiv.org/abs/2203.02155

[5] A. Rana, M. Oesterle, and J. Brinkmann, “Gov-rek: Governed reward engineering kernels for designing robust multi-agent reinforcement learning systems,” *arXiv*, 2024. [Online]. Available: http://arxiv.org/abs/2404.01131v2

[6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” *arXiv OpenAlex*, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[7] H. Yang, J. Wang, and X. Zhang, “Dica: Dual-indicator guided contrastive alignment in multimodal large language models,” *Crossref*, 2026. [Online]. Available: https://doi.org/10.18653/v1/2026.findings-acl.1933

[8] S. Hussain, M. Ali, F. A. Pirzado, M. Ahmed, and J. G. Tamez-Peña, "Comparative analysis of deep learning models for breast cancer classification on multimodal data," *Crossref*, 2024. [Online]. Available: https://doi.org/10.1145/3689096.3689462

[9] F. Bredell, H. A. Engelbrecht, and J. C. Schoeman, "Augmenting the action space with conventions to improve multi-agent cooperation in hanabi," *arXiv*, 2024. [Online]. Available: http://arxiv.org/abs/2412.06333v3

[10] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, and Z. Wang, "Metagpt: Meta programming for a multi-agent collaborative framework," *Crossref*, 2023. [Online]. Available: https://doi.org/10.48550/arxiv.2308.00352

[11] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, and W. Chen, "Chatdev: Communicative agents for software development," *Crossref*, 2024. [Online]. Available: https://doi.org/10.18653/v1/2024.acl-long.810

[12] J. Yang, C. Jimenez-Gomez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," *Crossref*, 2024. [Online]. Available: https://doi.org/10.48550/arxiv.2405.15793